

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5

8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I G Vyshnavi(192525260) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G Vyshnavi

1. Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

Relational Model

The School Management System consists of the following relations:

STUDENT (*StudentID, StudentName, ClassID, Grade*)

CLASS (*ClassID, ClassName, TeacherID*)

TEACHER (*TeacherID, TeacherName, Subject*)

- **Primary Keys:** StudentID, ClassID, TeacherID
- **Foreign Keys:** ClassID in Student, TeacherID in Class

Relational Algebra

$\pi_{\text{StudentName, TeacherName, Grade}}(\text{Student} \bowtie \text{Class} \bowtie \text{Teacher})$

This joins the Student, Class, and Teacher tables and displays the student name, teacher name, and grade.

Query:

```
SELECT s.StudentName,  
       t.TeacherName,  
       s.Grade  
FROM Student s  
JOIN Class c  
ON s.ClassID = c.ClassID  
JOIN Teacher t  
ON c.TeacherID = t.TeacherID;
```

View:

```
CREATE VIEW StudentTeacherView AS  
SELECT s.StudentName,  
       t.TeacherName,  
       s.Grade  
FROM Student s  
JOIN Class c  
ON s.ClassID = c.ClassID  
JOIN Teacher t  
ON c.TeacherID = t.TeacherID;  
SELECT * FROM StudentTeacherView;
```

Output:

```
mysql> SELECT * FROM Class;  
mysql> +-----+-----+-----+  
| ClassID | ClassName | TeacherName |  
+-----+-----+-----+  
| 101 | 10-A | Radhika |  
| 102 | 10-B | Suresh |  
| 103 | 9-A | Anitha |  
+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> -- STEP 7: DISPLAY STUDENT TABLE  
Query OK, 0 rows affected (0.00 sec)  
  
mysql> SELECT * FROM Student;  
mysql> +-----+-----+-----+-----+  
| SID | StudentName | Grade | ClassID |  
+-----+-----+-----+-----+  
| 1 | Rahul | A | 101 |  
| 2 | Priya | B | 102 |  
| 3 | Kiran | A | 101 |  
| 4 | Anjali | A+ | 103 |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> -- STEP 8: LIST ALL STUDENTS WITH THEIR CLASS TEACHER AND GRADE  
Query OK, 0 rows affected (0.00 sec)  
  
mysql> SELECT  
mysql> -> S.StudentName,  
mysql> -> C.ClassName,  
mysql> -> C.TeacherName,  
mysql> -> S.Grade  
mysql> -> FROM Student S  
mysql> -> INNER JOIN Class C  
mysql> -> ON S.ClassID = C.ClassID;  
mysql> +-----+-----+-----+-----+  
| StudentName | ClassName | TeacherName | Grade |  
+-----+-----+-----+-----+  
| Rahul | 10-A | Radhika | A |  
| Kiran | 10-A | Radhika | A |  
| Priya | 10-B | Suresh | B |  
| Anjali | 9-A | Anitha | A+ |  
+-----+-----+-----+-----+
```

2. A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

Scenario:

A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than **10 appointments in the current month**.

Relational Schema

DOCTOR (*DoctorID, DoctorName, Specialization*)

PATIENT (*PatientID, PatientName, Age*)

APPOINTMENT (*AppointmentID, PatientID, DoctorID, AppointmentDate*)

- **Primary Keys:** DoctorID, PatientID, AppointmentID
- **Foreign Keys:** PatientID and DoctorID in the Appointment table

Query:

```
SELECT d.DoctorID,
       d.DoctorName,
       COUNT(a.AppointmentID) AS TotalAppointments
FROM Doctor d
JOIN Appointment a
ON d.DoctorID = a.DoctorID
WHERE MONTH(a.AppointmentDate) = MONTH(CURDATE())
AND YEAR(a.AppointmentDate) = YEAR(CURDATE())
GROUP BY d.DoctorID, d.DoctorName
HAVING COUNT(a.AppointmentID) > 10;
```

Explanation

- The **Doctor**, **Patient**, and **Appointment** tables store hospital data.
- **JOIN** combines the Doctor and Appointment tables.
- **WHERE** filters appointments for the current month.
- **GROUP BY** groups records by doctor.
- **HAVING COUNT() > 10** displays only doctors who have more than 10 appointments.

Output:

```
mysql>
mysql> SELECT * FROM Appointment;
+-----+-----+-----+-----+
| AppointmentID | PatientID | DoctorID | AppointmentDate |
+-----+-----+-----+-----+
| 1001 | 1 | 101 | 2026-07-01 |
| 1002 | 2 | 101 | 2026-07-02 |
| 1003 | 3 | 101 | 2026-07-03 |
| 1004 | 4 | 101 | 2026-07-04 |
| 1005 | 5 | 101 | 2026-07-05 |
| 1006 | 1 | 101 | 2026-07-06 |
| 1007 | 2 | 101 | 2026-07-07 |
| 1008 | 3 | 101 | 2026-07-08 |
| 1009 | 4 | 101 | 2026-07-09 |
| 1010 | 5 | 101 | 2026-07-10 |
| 1011 | 1 | 101 | 2026-07-11 |
| 1012 | 2 | 102 | 2026-07-12 |
| 1013 | 3 | 103 | 2026-07-13 |
+-----+-----+-----+-----+
13 rows in set (0.00 sec)

mysql>
mysql> -- STEP 9: FIND DOCTORS WITH MORE THAN 10 APPOINTMENTS THIS MONTH
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT
  -> D.DoctorID,
  -> D.DoctorName,
  -> COUNT(A.AppointmentID) AS TotalAppointments
  -> FROM Doctor D
  -> JOIN Appointment A
  -> ON D.DoctorID = A.DoctorID
  -> WHERE MONTH(A.AppointmentDate) = 7
  -> AND YEAR(A.AppointmentDate) = 2026
  -> GROUP BY D.DoctorID, D.DoctorName
  -> HAVING COUNT(A.AppointmentID) > 10;
+-----+-----+-----+
| DoctorID | DoctorName | TotalAppointments |
+-----+-----+-----+
| 101 | Dr. Ravi | 11 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

3. In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

Scenario:

An e-commerce company wants to identify products that have **never been ordered**. Write an SQL query using a **sub-query** to find these products.

QRelational Schema

PRODUCT (*ProductID*, *ProductName*, *Price*)

ORDERS (*OrderID*, *CustomerID*, *OrderDate*)

ORDER_DETAILS (*OrderID*, *ProductID*, *Quantity*)

- **Primary Keys:** ProductID, OrderID
- **Foreign Keys:** OrderID and ProductID in **ORDER_DETAILS**

Query:

SELECT ProductID, ProductName

FROM Product

WHERE ProductID NOT IN (

SELECT ProductID

FROM Order_Details

);

Output:

```
+-----+-----+-----+-----+
| OrderID | ProductID | Quantity | OrderDate |
+-----+-----+-----+-----+
|      1  |      101  |        2 | 2026-07-10 |
|      2  |      102  |        5 | 2026-07-12 |
|      3  |      104  |        1 | 2026-07-15 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- STEP 8: FIND PRODUCTS THAT HAVE NEVER BEEN ORDERED (SUBQUERY)
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT ProductID, ProductName, Price
  -> FROM Product
  -> WHERE ProductID NOT IN
  -> (
  ->     SELECT ProductID
  ->     FROM Orders
  -> );
+-----+-----+-----+
| ProductID | ProductName | Price |
+-----+-----+-----+
|      103  | Keyboard    | 1200.00 |
|      105  | Printer     | 8000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> -- STEP 9: ALTERNATIVE USING NOT EXISTS
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT P.ProductID, P.ProductName, P.Price
  -> FROM Product P
  -> WHERE NOT EXISTS
  -> (
  ->     SELECT *
  ->     FROM Orders O
  ->     WHERE O.ProductID = P.ProductID
  -> );
+-----+-----+-----+
| ProductID | ProductName | Price |
+-----+-----+-----+
|      103  | Keyboard    | 1200.00 |
|      105  | Printer     | 8000.00 |
+-----+-----+-----+
```

4. For a university database, write relational algebra expressions to find students enrolled in both DBMS and OS courses.

Scenario:

A university wants to identify students who are enrolled in **both DBMS and OS courses**. Write the **relational algebra expression** to find these students.

Relations

STUDENT (*StudentID*, *StudentName*)

ENROLLMENT (*StudentID*, *CourseID*)

COURSE (*CourseID*, *CourseName*)

Relational Algebra Expression

Let,

- **A** = Students enrolled in **DBMS**
- **B** = Students enrolled in **OS**

$A = \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='DBMS'}(\text{ENROLLMENT} \bowtie \text{COURSE}))$
 $A = \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='DBMS'}(\text{ENROLLMENT} \bowtie \text{COURSE}))$
 $B = \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='OS'}(\text{ENROLLMENT} \bowtie \text{COURSE}))$
 $B =$

$\pi_{\{StudentID\}}(\sigma_{\{CourseName='OS'\}}(ENROLLMENT \bowtie COURSE)) \cap \pi_{\{StudentID\}}(\sigma_{\{CourseName='DBMS'\}}(ENROLLMENT \bowtie COURSE))$

Students enrolled in **both** courses:

$A \cap B$

Explanation

- **Selection (σ)** filters the DBMS and OS courses.
- **Join ($\bowtie$)** combines the Enrollment and Course tables.
- **Projection (π)** retrieves only the StudentID.
- **Intersection ($\cap$)** returns students who are enrolled in both **DBMS** and **OS**.

Output:

```
mysql>
mysql> SELECT S.Name
-> FROM Student S
-> JOIN Enrollment E1 ON S.SID=E1.SID
-> JOIN Course C1 ON E1.CourseID=C1.CourseID
-> JOIN Enrollment E2 ON S.SID=E2.SID
-> JOIN Course C2 ON E2.CourseID=C2.CourseID
-> WHERE C1.CourseName='DBMS '
-> AND C2.CourseName='OS ' ;
+-----+
| Name |
+-----+
| Rahul |
| Kiran |
+-----+
2 rows in set (0.00 sec)
```

5. Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

Scenario:

A company wants to identify employees whose salary is **below their department's average salary**. Create a **view** for these employees and write an SQL query to increase their salary by 10%.

Relations

EMPLOYEE (*EmpID, EmpName, DeptID, Salary*)

DEPARTMENT (*DeptID, DeptName*)

Query:

CREATE VIEW BelowAvgSalary AS

SELECT e.EmpID, e.EmpName, e.DeptID, e.Salary

FROM Employee e

WHERE e.Salary < (

SELECT AVG(Salary)

FROM Employee

WHERE DeptID = e.DeptID

);

Output:

```
mysql>
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Rahul   | 1       | 55000.00 |
| 102   | Priya   | 1       | 70000.00 |
| 103   | Kiran   | 1       | 49500.00 |
| 104   | Anjali  | 2       | 44000.00 |
| 105   | Ravi    | 2       | 60000.00 |
| 106   | Sneha   | 3       | 55000.00 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

Scenario:

A bank wants to retrieve **customer account details, transaction history, and branch information**. Write an SQL query using **JOIN** operations.

Relations

CUSTOMER (*CustomerID, CustomerName*)

ACCOUNT (*AccountNo, CustomerID, BranchID, Balance*)

TRANSACTION (*TransactionID, AccountNo, Amount, TransactionDate*)

BRANCH (*BranchID, BranchName, BranchCity*)

Query:

SELECT c.CustomerName,

a.AccountNo,

t.TransactionID,

t.Amount,

t.TransactionDate,

b.BranchName,

b.BranchCity

FROM Customer c

JOIN Account a

ON c.CustomerID = a.CustomerID

JOIN Transaction t

ON a.AccountNo = t.AccountNo

JOIN Branch b

ON a.BranchID = b.BranchID;

Output:

```
-> ON A.AccountNo = T.AccountNo;
```

CustomerName	BranchName	City	AccountNo	TransactionType	Amount
Rahul	SBI Main Branch	Hyderabad	10001	Deposit	10000.00
Rahul	SBI Main Branch	Hyderabad	10001	Withdrawal	5000.00
Kiran	SBI Main Branch	Hyderabad	10003	Withdrawal	3000.00
Priya	SBI City Branch	Chennai	10002	Deposit	20000.00
Anjali	SBI Central Branch	Bangalore	10004	Deposit	15000.00

5 rows in set (0.00 sec)

7. A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

Scenario:

A retail store wants to generate a **daily sales summary**. Write an SQL query using **GROUP BY** and **HAVING** to display the **total sales for each category** where the total sales exceed **Rs. 10,000**.

Relations

PRODUCT (*ProductID, ProductName, Category*)

SALES (*SaleID, ProductID, SaleDate, Amount*)

Query:

SELECT p.Category,

SUM(s.Amount) AS TotalSales

FROM Product p

JOIN Sales s

ON p.ProductID = s.ProductID

WHERE SaleDate = CURDATE()

GROUP BY p.Category

HAVING SUM(s.Amount) > 10000;

Output:

```
mysql>
mysql> SELECT * FROM Sales;
+-----+-----+-----+-----+
| SaleID | ProductID | Quantity | SaleDate |
+-----+-----+-----+-----+
| 1      | 101       | 1        | 2026-07-30 |
| 2      | 102       | 10       | 2026-07-30 |
| 3      | 103       | 2        | 2026-07-30 |
| 4      | 104       | 1        | 2026-07-30 |
| 5      | 105       | 20       | 2026-07-30 |
| 6      | 106       | 1        | 2026-07-30 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql> -- STEP 8: TOTAL SALES PER CATEGORY EXCEEDING Rs.10000
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT
  -> P.Category,
  -> SUM(P.Price * S.Quantity) AS TotalSales
  -> FROM Product P
  -> JOIN Sales S
  -> ON P.ProductID = S.ProductID
  -> GROUP BY P.Category
  -> HAVING SUM(P.Price * S.Quantity) > 10000;
+-----+-----+
| Category | TotalSales |
+-----+-----+
| Electronics | 67000.00 |
| Furniture | 13000.00 |
+-----+-----+
2 rows in set (0.00 sec)
```

8. For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

Scenario:

A library wants to identify **members who have borrowed books from all available genres**. Write a **Tuple Relational Calculus (TRC)** expression to find these members.

Relations

MEMBER (*MemberID*, *MemberName*)

BOOK (*BookID*, *Title*, *Genre*)

BORROW (*MemberID*, *BookID*, *BorrowDate*)

Tuple Relational Calculus (TRC)

$\{ m.MemberName \mid MEMBER(m) \wedge \forall g (BOOK(g) \rightarrow \exists b (BORROW(b) \wedge b.MemberID = m.MemberID \wedge b.BookID = g.BookID)) \}$

$\{ \mid m.MemberName \mid MEMBER(m) \wedge \forall g \setminus, (BOOK(g) \rightarrow \exists b \setminus, (BORROW(b) \wedge b.MemberID = m.MemberID \wedge b.BookID = g.BookID)) \}$

Explanation

- **MEMBER** contains member details.
- **BOOK** contains books with different genres.
- **BORROW** stores borrowing records.
- The expression retrieves members who have borrowed books covering **all available genres**.

Output:

```
mysql> SELECT * FROM Borrow;
mysql>
+-----+-----+-----+-----+
| BorrowID | MemberID | BookID | BorrowDate |
+-----+-----+-----+-----+
| 1 | 1 | 101 | 2026-07-01 |
| 2 | 1 | 102 | 2026-07-02 |
| 3 | 1 | 103 | 2026-07-03 |
| 4 | 2 | 101 | 2026-07-04 |
| 5 | 2 | 102 | 2026-07-05 |
| 6 | 3 | 101 | 2026-07-06 |
| 7 | 3 | 102 | 2026-07-07 |
| 8 | 3 | 103 | 2026-07-08 |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql>
mysql> -- STEP 9: MEMBERS WHO BORROWED BOOKS FROM ALL GENRES
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT M.MemberName
mysql> FROM Member M
mysql> WHERE NOT EXISTS
mysql> (
mysql> SELECT DISTINCT Genre
mysql> FROM Book B
mysql> WHERE NOT EXISTS
mysql> (
mysql> SELECT *
mysql> FROM Borrow BR
mysql> JOIN Book BK
mysql> ON BR.BookID = BK.BookID
mysql> WHERE BR.MemberID = M.MemberID
mysql> AND BK.Genre = B.Genre
mysql> )
mysql> );
+-----+
| MemberName |
+-----+
| Rahul      |
| Kiran      |
+-----+
2 rows in set (0.00 sec)
```

9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

Scenario:

An inventory management system needs to maintain valid product data. Define **CHECK** and **FOREIGN KEY** constraints and demonstrate their enforcement using SQL.

Relations

CATEGORY (*CategoryID*, *CategoryName*)

PRODUCT (*ProductID*, *ProductName*, *CategoryID*, *Price*, *Quantity*)

Query:

CREATE TABLE Category (

CategoryID INT PRIMARY KEY,

CategoryName VARCHAR(50)

);

CREATE TABLE Product (

ProductID INT PRIMARY KEY,

ProductName VARCHAR(50),

CategoryID INT,

Price DECIMAL(10,2) CHECK (Price > 0),

Quantity INT CHECK (Quantity >= 0),

FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)

);

Output:

```
mysql> Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Product;
+-----+-----+-----+-----+-----+
| ProductID | ProductName | Price   | Quantity | SupplierID |
+-----+-----+-----+-----+-----+
| 1 | Laptop | 50000.00 | 20 | 101 |
| 2 | Mouse | 500.00 | 100 | 101 |
| 3 | Keyboard | 1200.00 | 50 | 102 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

10.Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

Scenario:

An HR system needs to maintain employee records and generate **monthly attendance** and **performance summaries**. Design the relational schema and create **materialized views**.

Relational Schema

EMPLOYEE (*EmpID, EmpName, DeptID*)

ATTENDANCE (*EmpID, Month, DaysPresent*)

PERFORMANCE (*EmpID, Month, Rating*)

Query:

CREATE MATERIALIZED VIEW MonthlyAttendance AS

SELECT EmpID,

Month,

SUM(DaysPresent) AS TotalDays

FROM Attendance

GROUP BY EmpID, Month;CREATE MATERIALIZED VIEW PerformanceSummary AS

SELECT EmpID,

Month,

AVG(Rating) AS AvgRating

FROM Performance

GROUP BY EmpID, Month;

Output:

```
mysql>
mysql> SELECT * FROM MonthlyAttendance;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | Department | Month | DaysPresent |
+-----+-----+-----+-----+-----+
| 101   | Rahul   | IT         | July  | 26          |
| 102   | Priya   | HR         | July  | 24          |
| 103   | Kiran   | Finance    | July  | 27          |
| 104   | Anjali  | IT         | July  | 25          |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- STEP 11: CREATE VIEW FOR PERFORMANCE SUMMARY
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> CREATE VIEW PerformanceSummary AS
-> SELECT
-> E.EmpID,
-> E.EmpName,
-> E.Department,
-> P.Rating
-> FROM Employee E
-> JOIN Performance P
-> ON E.EmpID = P.EmpID;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- STEP 12: DISPLAY PERFORMANCE SUMMARY VIEW
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM PerformanceSummary;
+-----+-----+-----+-----+
| EmpID | EmpName | Department | Rating |
+-----+-----+-----+-----+
| 101   | Rahul   | IT         | 4.5    |
| 102   | Priya   | HR         | 4.0    |
| 103   | Kiran   | Finance    | 4.8    |
| 104   | Anjali  | IT         | 4.2    |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```